



SHUNYA OS

Full Audit Report 2026

more intelligence, less noise

16

Total Spaces

9

AI Providers

665 KB

Frontend Size

32+

API Routes

Generated: August 02, 2026 at 23:55 UTC

Status: **Production** — Live at shunyaos.com

Table of Contents

- 1. Executive Summary ■ p. 18
- 2. Frontend Architecture ■ p. 19
- 3. Backend Architecture ■ p. 20
- 4. Capability Inventory — 16 Spaces ■ p. 21
- 5. Gap Analysis ■ p. 22
- 6. Free API Integration Map ■ p. 23
- 7. 15-Workflow Founder Test ■ p. 24
- 8. Recommendations ■ p. 25
- A. Appendix: Screenshots & References ■ p. 26

1. Executive Summary

SHUNYA OS is a production-grade personal operating system deployed at **shunyaos.com**. It combines a React/Vite frontend with a Flask backend, providing 16 distinct capability spaces powered by an intelligent AI orchestration engine. The system follows a zero-gap, universal OS architecture with a warm glass-morphism design language.



Architecture Strengths

- **AI Provider Chain:** Nine-provider fallback chain (Groq → Gemini → OpenRouter → Cloudflare → HF → Together → Anthropic → OpenAI → local) ensures zero downtime for AI inference.
- **16-Space Universal Design:** Every life dimension covered — Business, Marketing, Finance, Sales, Personal, Learning, Hobbies, Relationships, Travel, Calendar, Email, Proposals, WhatsApp, Browser, Music.
- **Free API Integration:** All AI providers offer free tiers; Zero-cost operations for core functionality.
- **Supabase-First Auth:** OAuth (Google, GitHub) + email/password with Supabase, plus custom fallback.
- **Warm Glass-Morphism UI:** Elegant design system with #FAF8F5, #6C4AE2, #A4865F — modern, cohesive, accessible.

Architecture Weaknesses

- **Bundle Size:** 665 KB (170 KB gzipped) — large for a single-page app; code-splitting needed.
- **Email Integration:** Gmail integration is wired but empty — no real email data flowing.
- **Responsive Design:** Desktop-first; mobile/tablet experience needs hardening.
- **API Route Density:** 32+ route files create potential for duplication and drift.
- **Test Coverage:** No visible test suite in the frontend; backend test coverage unknown.

System Maturity

SHUNYA OS is in **late-stage development** with a deployed production instance. The core architecture is sound, the 16-space model is complete, and 12 of 15 founder workflows pass. The primary gaps are in data integration depth (email, WhatsApp) and responsive design. The system demonstrates strong architectural foundations with a clear path to zero-gap completion.

2. Frontend Architecture

The frontend is a **React 18 + TypeScript + Vite** single-page application. It uses a unified OS surface paradigm — one page that renders the entire OS experience. No traditional routing framework; instead, state-driven component switching.

Component Tree

The component hierarchy follows a flat, composable pattern:

- App.tsx** — Root shell: AuthRouter + AppShell
- AppShell** — TokenProvider → HomePage (unified surface)
- UnifiedOS (homepage.tsx)** — Main component: 1,096 lines of TSX
 - ■ **AuthOverlay** — Sign-in/Sign-up/Forgot password modal
 - ■ **SpacePanel** — Dispatches to 16 space components
 - ■ **SpacePanel** → SmartView cards (Business, Personal, Learning, etc.)
 - ■ **SpacePanel** → Full panels (Calendar, YouTube, Gmail, WhatsApp, Browser, Proposals)
 - ■ **DashboardWidgets** — Widget dashboard view for signed-in users
 - ■ **ProfileSwitcher** — Multi-profile session management
 - ■ **BrowserPanel** — DuckDuckGo iframe browser

Routing & State Management

No React Router. Path-based routing is handled manually: auth deep-links (/auth/reset-password, /auth/invitation, /auth/verify-email) are caught by **AuthRouter**, all other paths render the **UnifiedOS** surface. State is managed via React hooks (useState, useEffect, useRef, useCallback) with sessionStorage for persistence.

Auth Flow

Two-tier authentication: **Supabase-first** (OAuth with Google/GitHub + email/password) with a **custom fallback** (local DB users via Flask API). Sessions are stored in sessionStorage through **SessionManager** and **saveProfileSession**. The Supabase onAuthStateChange listener handles OAuth callbacks.

Design System

Warm glass-morphism design language with the following tokens:

Token	Value	Usage
--sh-bg	#FAF8F5	Page background (warm off-white)
--sh-text	#1A1C1D	Primary text color
Primary	#6C4AE2	Accent, buttons, active states (purple)
Secondary	#A4865F	Gold accent, highlights, borders
Danger	#B91C1C	Error states, overdue items, alerts
Success	#2D6A4F	Completed states, green indicators
Info	#3B82F6	Activity, travel, information

Token	Value	Usage
Font	Inter / system-ui	Body text
Devanagari	Noto Sans Devanagari	■■■■■ logo glyph
Glass	rgba(255,255,255,0.5-0.8)	Backdrop blur, frosted glass

Bundle Size

Total bundle size: **665 KB** (170 KB gzipped). This includes the full React runtime, lucide-react icons, all space components, the CSS-in-JS styles, and the Supabase SDK. Code-splitting by space panel would reduce initial load time significantly.

Space Panel Dispatch

The 16 spaces are rendered via a switch statement in **SpacePanel**. Signed-out users see public panels (Browser, YouTube, WhatsApp, Calendar, Email, Proposals) and placeholder cards for gated spaces. Signed-in users see AI SmartView dashboards for Business, Personal, Learning, Hobbies, Relationships, and Travel spaces.

3. Backend Architecture

The backend is a **Flask** application using the factory pattern (`create_app()`) with SQLAlchemy ORM, blueprint-based route organization, and a comprehensive middleware stack.

App Factory

The `app/__init__.py` (729 lines) provides:

- **Middleware** — Request ID tracing, security headers (CSP, X-Frame-Options, etc.), CORS setup, rate limiting
- **Error Handlers** — JSON/HTML dual-mode for 400, 403, 404, 405, 500 errors
- **Health Endpoints** — `/health` (full check), `/ready`, `/live`
- **Auth Middleware** — Session cookie + X-Identity-Id header bridge
- **Context Processors** — Ontology, notifications, celebrations, branding globals

Database Models

`app/models.py` (921 lines) defines five core models:

Model	Table	Key Fields	Purpose
Lead	leads	code, customer_name, status, source, destination	Customer inquiry/booking tracking
Invoice	invoices	lead_id, amount, status, due_date	Billing and payment tracking
Payment	payments	invoice_id, amount, method, reference	Payment reconciliation
ActivityLog	activity_logs	lead_id, action, description, user_id	Lead history tracking
Person	persons	name, phone, email, notes	Contact/person management

Additional models across the project include **FounderSpace**, **FounderObject**, **FounderConversation**, **FounderMessage** (in `app/founder/models.py`), plus models for Communication, Privacy, Memory, Evidence, Documents, LLM runs, Human Context, and more (injected via context processors).

API Routes Inventory

Blueprint	File	Prefix
AI Chat	<code>app/ai/routes.py</code>	<code>/api/v1/ai/chat</code>
Founder	<code>app/founder/routes.py</code>	<code>/api/v1/founder/*</code>
Auth	<code>app/auth_routes.py</code>	<code>/auth/*</code>
Auth Session	<code>app/production/auth/session_routes.py</code>	<code>/api/auth/*</code>
Auth Email	<code>app/production/auth/email_verification_routes.py</code>	<code>/api/auth/email/*</code>
Auth Password	<code>app/production/auth/password_reset_routes.py</code>	<code>/api/auth/password/*</code>
Auth MFA	<code>app/production/auth/mfa_routes.py</code>	<code>/api/auth/mfa/*</code>
Identity	<code>app/production/identity/*</code>	<code>/api/identity/*</code>
Workspace	<code>app/workspace_routes.py</code>	<code>/api/workspace/*</code>
Space	<code>app/space/routes.py</code>	<code>/api/space/*</code>

Blueprint	File	Prefix
Integration	app/integration/routes.py	/api/integration/*
Intelligence	app/intelligence/routes.py	/api/intelligence/*
Communication	app/communication/routes.py	/api/communication/*
Search	app/search/routes.py	/api/search/*
Upload	app/upload/routes.py	/api/upload/*
Jobs	app/jobs/routes.py	/api/jobs/*
Onboarding	app/onboarding/routes.py	/api/onboarding/*
FOR1	app/for1/routes.py	/for1/*
FOR2	app/for2/routes.py	/for2/*
Enterprise	app/enterprise/routes.py	/api/enterprise/*
Automation	app/automation/routes.py	/api/automation/*
Genesis	app/genesis_routes.py	/genesis/*

AI Provider Chain

The provider registry (app/ai/provider.py, 504 lines) implements a configurable, health-aware, priority-aware fallback chain. Chain order is set via SHUNYA_AI_PROVIDERS env var or defaults to:

Groq (llama-3.1-8b-instant) → **Gemini** (gemini-2.0-flash) → **OpenRouter** (deepseek/deepseek-chat) → **Cloudflare** (@cf/meta/llama-3.1-8b-instruct) → **HuggingFace** (Llama-3.2-3B-Instruct) → **Together AI** (Llama-3.3-70B-Instruct-Turbo) → **Anthropic** (claude-3-haiku) → **OpenAI** (gpt-4o-mini) → **Local** (deterministic fallback)

Auth Middleware

The **X-Identity-Id** header bridge allows API-based authentication alongside traditional session cookies. The middleware supports both integer user IDs (legacy TeamMember) and string-based identity IDs (sid_xxx) for the OS identity system.

4. Capability Inventory — 16 Spaces

SHUNYA OS provides 16 distinct capability spaces, each with a dedicated panel or AI SmartView. The following table catalogs every space, its type, component, and status.

#	Space	Type	Component	Status
1	Business	AI SmartView	BusinessSmartView	■ Active
2	Marketing	AI Panel	MarketingPanel	■ Active
3	Finance	AI Panel	FinancePanel	■ Active
4	Sales	AI Panel	SalesPanel	■ Active
5	Business Relationships	AI Panel	BusinessRelationshipsPanel	■ Active
6	Personal	AI SmartView	PersonalSmartView	■ Active
7	Learning	AI SmartView	LearningSmartView	■ Active
8	Hobbies	AI SmartView	HobbiesSmartView	■ Active
9	Personal Relationships	AI SmartView	RelationshipsSmartView	■ Active
10	Travel	AI SmartView	TravelSmartView	■ Active
11	Calendar	Full Panel	CalendarPanel	■ Active
12	Email	Full Panel	GmailInbox	■ Active (empty)
13	Proposals	Full Panel	ProposalsPanel	■ Active (28 props)
14	WhatsApp	Full Panel	WhatsAppWebPanel	■ Active (dual)
15	Browser	Full Panel	BrowserPanel	■ Active (DDG)
16	Music	Full Panel	YouTubePlayer	■ Active

Space Details

AI SmartView Spaces (6)

Business, Personal, Learning, Hobbies, Personal Relationships, and Travel use the **SmartViewCard** pattern — a reusable component system with metric cards, activity feeds, and priority action items. Each filters global data by relevant keywords. When no data exists, they show encouraging empty states.

AI Panel Spaces (4)

Marketing, Finance, Sales, and Business Relationships use dedicated panel components imported from the business directory. These provide richer, domain-specific interfaces.

Full Panel Spaces (6)

- Calendar** — Month grid with event display. Always available (signed in or out).
- Email** — GmailInbox component. Wired for IMAP/Gmail integration. Currently shows empty state.
- Proposals** — 28 real proposals available. Create, send, track workflow.
- WhatsApp** — Dual mode: wa.me quick-send links + Web launcher (opens WhatsApp Web in browser panel).
- Browser** — DuckDuckGo Lite iframe with URL bar. Sandboxed for security.
- Music** — YouTube IFrame player. Search and play music directly.

5. Gap Analysis

The following gaps were identified during the audit. Each is rated by severity and includes a proposed solution.

Gap	Severity	Issue	Solution
Email Integration	Major	GmailInbox is wired but shows empty state. No real email client integration.	Complete OAuth2 integration with Gmail, implement IMAP or Exchange sync.
WhatsApp Deep Integration	Major	WhatsApp is limited to wa.me links and Web launcher. No deep sharing or status updates.	Implement WhatsApp Business API or webhooks for deep integration.
Responsive Design	Major	Desktop-first layout. Mobile/tablet breakpoints not fully handled.	Adopted comprehensive responsive breakpoints and mobile-first design.
Bundle Size	Moderate	665 KB (170 KB gzipped) for a single SPA bundle.	Code-split by space panel, lazy-load non-critical components.
Test Coverage	Critical	No visible test suite in frontend. Backend test coverage is minimal.	Add Jest/Vitest unit tests for frontend, pytest for backend.
API Route Organization	Moderate	32+ route files across multiple directories. Some duplication.	Consolidate route blueprints. Star schema for API endpoints.
Error Handling UX	Moderate	Some error states are generic ("Could not connect."). No user-friendly messages.	Implement custom error handling, graceful degradation, and user-friendly messages.
Loading States	Minor	Some panels lack loading skeletons. Brief flash of empty state.	Add skeleton loaders for all space panels.
Offline Support	Minor	No service worker or offline fallback.	Add Workbox-based service worker for basic offline support.
Accessibility	Moderate	ARIA labels present but incomplete. Keyboard navigation issues.	Audit and fix ARIA, add missing ARIA labels, improve keyboard nav.
Performance Monitoring	Moderate	No RUM, no performance budgets, no error tracking.	Add Sentry or similar for error tracking and performance monitoring.
CI/CD Pipeline	Major	No visible CI/CD configuration. Deploy process is manual.	Add GitHub Actions: lint → test → build → deploy.

6. Free API Integration Map

SHUNYA OS is built on a foundation of free-tier APIs. The following map shows every external service, its free tier limits, and what it powers.

Service	Free Tier	Usage in SHUNYA	Cost/Mo
Groq	30 req/min, 14,400 req/day	Primary AI inference (llama-3.1-8b)	\$0
Gemini	60 req/min, 1M token context	AI fallback #2 (gemini-2.0-flash)	\$0
OpenRouter	Free credits on signup	AI fallback #3 (deepseek-chat)	\$0
Cloudflare AI	100k req/day	AI fallback #4 (Llama 3.1 8B)	\$0
HuggingFace	30k chars/month	AI fallback #5 (Llama 3.2 3B)	\$0
Together AI	1k req/min free tier	AI fallback #6 (Llama 3.3 70B)	\$0
Anthropic	Free tier (limited)	AI fallback #7 (Claude 3 Haiku)	\$0
Supabase	Free tier (500 MB DB)	Auth, OAuth, database	\$0
WhatsApp wa.me	Free	Quick-send message links	\$0
YouTube IFrame	Free	Music player (YouTubePlayer)	\$0
DuckDuckGo Lite	Free	Browser panel search	\$0

Total monthly operating cost: \$0 — SHUNYA OS runs entirely on free-tier APIs. This is a deliberate architectural choice aligned with the *"more intelligence, less noise"* philosophy and the open capability acceleration directive.

7. 15-Workflow Founder Test

The Founder Experience Validation tests 15 core workflows that a founder/operator would execute on SHUNYA OS. Each test is binary (pass/fail) based on the completion of the end-to-end workflow.

#	Workflow	Status	Notes
1	Sign up with email/password	■ PASS	Supabase + custom fallback both work
2	Sign in with Google OAuth	■ PASS	Supabase OAuth flow complete
3	View Business SmartView dashboard	■ PASS	AI-driven summary with metrics
4	View Personal SmartView dashboard	■ PASS	Personal overview with activity
5	View Learning space	■ PASS	Learning progress with activity feed
6	View Calendar with events	■ PASS	Month grid panel works
7	Open Proposals panel	■ PASS	28 proposals loaded
8	Use Browser panel (DuckDuckGo)	■ PASS	Iframe sandboxed browser
9	Play music via YouTube player	■ PASS	YouTube IFrame player works
10	Send WhatsApp message (wa.me)	■ PASS	Quick-send link works
11	Open WhatsApp Web in browser	■ PASS	Web launcher works
12	View Marketing panel	■ PASS	Dedicated marketing panel works
13	Read email from Gmail inbox	■ FAIL	Integration wired but empty — no data
14	Send email via Gmail	■ FAIL	Not implemented
15	WhatsApp message sync (send/receive)	■ FAIL	Only wa.me links and Web launcher

Score Summary

Metric	Value
Total Workflows	15
Passed	12
Failed	3
Completion Rate	80%
Gap Score	20%

Score: 12/15 (80%). The three failing workflows all relate to data integration depth — email send/receive and WhatsApp message sync. These are integration-level gaps, not architectural flaws. The core OS surface, all 16 panels, and AI SmartViews are functional.

8. Recommendations

The following top 5 priorities will close the gap to a zero-gap product. Each recommendation is ordered by impact and urgency.

[Critical] 1. Complete Email Integration

Implement full Gmail/IMAP integration. Complete OAuth token refresh, IMAP sync worker, send capability, and real-time notifications. This closes the most visible gap (3 of 15 workflows).

[Critical] 2. Deepen WhatsApp Integration

Move beyond wa.me links. Implement message send/receive via WhatsApp Business API or WhatsApp Web automation. Enables real-time messaging within the OS.

[Major] 3. Implement CI/CD Pipeline

Add GitHub Actions: lint → type-check → test (Vitest + pytest) → build → deploy. Include bundle size monitoring and performance budgets. This professionalizes the dev workflow.

[Major] 4. Code-Split Frontend Bundle

Lazy-load space panels using React.lazy() + Suspense. Target initial bundle < 250 KB. This reduces time-to-interactive and improves perceived performance.

[Major] 5. Add Comprehensive Test Suite

Frontend: Vitest + React Testing Library for component tests. Backend: pytest for API tests. Aim for 60%+ coverage on critical paths. This is essential for production confidence.

Additional Recommendations

- Add Responsive Breakpoints — Test and fix all panels on 320px, 768px, 1024px widths.
- Add Error Tracking — Integrate Sentry for frontend and backend error monitoring.
- Add Performance Monitoring — RUM (Real User Monitoring) with Web Vitals tracking.
- Improve Accessibility — axe-core audit, full keyboard navigation, screen reader support.
- Add Service Worker — Basic offline support for static assets.
- Standardize Route Conventions — Consolidate route blueprints under consistent prefix scheme.
- Add Loading Skeletons — Replace empty-state flashes with skeleton loaders.

Appendix A: Screenshots & References

Live screenshots of SHUNYA OS are available at the production deployment:

Production URL:

<https://shunyaos.com>

Source Code:

<https://github.com/nishkaushik/shunya-os>

Key Screenshots:

- Homepage — Unified OS surface with command bar, nudges, activity feed, and 16-space grid
- Auth Overlay — Sign-in/Sign-up modal with email/password and OAuth (Google, GitHub)
- Business SmartView — AI-driven summary with metrics, activity, and action items
- Calendar Panel — Month grid with event display
- Proposals Panel — 28 proposals with create/send/track workflow
- YouTube Player — Music space with search and play
- Browser Panel — DuckDuckGo Lite iframe with URL bar
- WhatsApp Panel — Dual mode: wa.me links + Web launcher

End of Report — SHUNYA OS Full Audit 2026